

Human↔AI Translator Platform

0) Executive Summary (Abstract)

Most people struggle to get high-quality results from AI models because they don't know *how to prompt*, *which model to pick*, or *how to judge outputs*. We propose a **Human↔AI Translator** that converts plain, messy user intent into a precise, structured **TaskSpec** and then auto-generates optimal prompts, routes the task to an appropriate model (based on task type, cost, latency, and quality expectations), and evaluates the results against user-defined acceptance criteria. The platform includes a **learning layer** that teaches users *why* certain prompts and models work better, turning everyday users into confident AI operators.

Outcome: A single interface for (1) intent → prompt compilation, (2) model selection guidance with transparent trade-offs, and (3) continuous learning via interactive lessons and real examples.

1) Problem Statement (Enhanced)

Original pain: - Lay users type vague requests (e.g., “Make a marketing plan”) and receive inconsistent outputs. - Users don't know *which* model is best for coding, writing, data analysis, image generation, etc. - Trial-and-error wastes tokens, time, and motivation; there's no systematic way to learn.

Underlying gaps: 1. **Intent Formalization Gap:** Natural language intentions are unstructured; models need task goals, constraints, style, references, and acceptance criteria. 2. **Model Fit Gap:** Different models excel at different tasks (e.g., code synthesis vs. long-form narrative vs. tool-use). Users lack clear, dynamic guidance. 3. **Quality Control Gap:** Users rarely specify evaluation rubrics, so they can't tell if outputs are “good enough” without manual rework. 4. **Learning Gap:** Prompting is tacit knowledge. There's no built-in feedback loop that explains *why* a prompt worked and how to improve it next time.

Problem Statement (final):

Build a platform that *translates human intent* into a structured TaskSpec, *compiles* it into an optimized prompt, *routes* it to the best-fit model under user-chosen constraints (quality, cost, latency, privacy), and *verifies* results against explicit acceptance criteria—while *teaching* the user the reasoning behind prompt/model choices.

2) Project Objectives

1. **Accuracy:** Consistently produce outputs that match user intent and acceptance criteria.
 2. **Usability:** Enable non-experts to get expert-grade results via a guided UI.
 3. **Transparency:** Show why a model/prompt was chosen, with cost/latency/quality trade-offs.
 4. **Learning:** Provide bite-sized lessons and inline explanations that improve user skill.
 5. **Extensibility:** Support multiple providers and local models via a single compatibility layer.
 6. **Safety & Privacy:** Enforce guardrails; respect user data boundaries; enable local/air-gapped mode.
-

3) Scope (MVP → V1)

In-Scope (MVP)

- Intent capture wizard → TaskSpec (goal, inputs, constraints, style, tone, audience, format, acceptance criteria).
- Prompt compiler with templates per task family (write, code, analyze, plan, translate, summarize, query-RAG, etc.).
- Model router with user-selectable preferences (cost/latency/quality/privacy) and default smart recommendations.
- Result verifier (LLM-as-judge + rule checks) against acceptance criteria.
- Learn layer: inline hints (“Why this prompt works”), short lessons, and model capability explainer.
- History & versioning with “diff” and one-click improvement loops.

Out-of-Scope (MVP) but planned for V1

- Team workspace, shared templates, and governance.
 - Advanced analytics: win-rate by task, cost dashboards, latency heatmaps.
 - API/SDK for third-party integrations; browser extension.
 - Human-in-the-loop labeling tools for building evaluation datasets.
-

4) Literature & Landscape (Brief)

- Prompt engineering best practices (task decomposition, role priming, examples, constraints).
- Model benchmarking across tasks (code, reasoning, writing, tool use); dynamic landscape → design the router to be data-driven and replaceable.
- Evaluation approaches: pairwise preference (A/B), rubric-based grading (LLM-as-judge), task-specific metrics (pass@k for code, factuality checks for RAG, etc.).

(Note: For the academic report, cite standard benchmarks and evaluation methods; avoid vendor-specific claims in the core design.)

5) User Personas & Jobs-to-Be-Done

1. **Student/Professional (Non-technical):** “I want a clear document/presentation/report without learning prompt tricks.”
2. **Developer/Data Analyst:** “I need code snippets, refactoring, test generation, or quick data analysis with guardrails.”
3. **Creator/Marketer:** “I want on-brand content drafted fast with consistent voice and target audience.”
4. **Manager/Founder:** “I need structured plans, decision briefs, and comparison matrices with transparent assumptions.”

JTBD: Turn a fuzzy goal → a solid deliverable, with the system doing the prompting, model choice, and QC.

6) Functional Requirements

- **FR-1 Intent Wizard:** Collects goal, context, inputs (files/links), constraints, tone/style, audience, output format, acceptance criteria.
 - **FR-2 TaskSpec Builder:** Normalizes inputs into a JSON schema (see §10).
 - **FR-3 Prompt Compiler:** Uses templates and examples per task family; injects TaskSpec; produces system/instructions, tools, and user prompts.
 - **FR-4 Model Router:** Selects model based on task type, user constraints, offline benchmark scores, and live performance telemetry; supports manual override.
 - **FR-5 Execution Engine:** Calls model/toolchain, streams output, handles retries, timeouts, and cost tracking.
 - **FR-6 Result Verifier:** Grades output vs acceptance criteria; flags gaps; proposes auto-fix loop ("Repair & Re-ask").
 - **FR-7 Learn Layer:** Inline "why it works" cards, short lessons, and model capability sheets.
 - **FR-8 History & Audit:** Versioned specs/prompts/outputs; reproducible runs.
 - **FR-9 Safety:** Input/content moderation, PII minimization, model/tool sandboxing, configurable data retention.
-

7) Non-Functional Requirements

- **Performance:** P95 end-to-end under user-set budgets (e.g., <5s for simple drafts; <15s for code/run loops).
 - **Reliability:** Retries with exponential backoff; circuit breakers per provider; idempotent runs.
 - **Scalability:** Stateless API + queue workers; horizontal scale; vector store decoupled.
 - **Security & Privacy:** OAuth for providers; SOC2-style controls; option for local models; no training on user data by default.
 - **Observability:** Traces per run (prompt, tokens, latency, cost, model); feature flags; evaluation dashboards.
-

8) System Architecture (High-Level)

Core services: 1. **Intent Service** (Wizard + NLU): extracts entities, goals, constraints → TaskSpec. 2. **Compiler Service:** compiles TaskSpec into prompt bundles per task family. 3. **Router Service:** chooses model using rules + multi-armed bandit (UCB/Thompson) with constraints (cost, latency, privacy, quality target). 4. **Execution Service:** provider SDK abstraction; tool invocation (web search, code exec, RAG, etc.). 5. **Verifier Service:** rubric grading (LLM-as-judge), rule checks (schema/length/coverage), hallucination checks for RAG. 6. **Learn Service:** generates "explainers," collects examples, and powers the Learn Center. 7. **Data Layer:** Postgres (runs, specs, telemetry), object store (artifacts), vector DB for RAG/lessons.

Tech stack (suggested): - Backend: **FastAPI** (Python) or **Node + NestJS**; workers via **Celery/RQ** or **BullMQ**. - Orchestration: **DSPy** or **Guidance** (prompt programs); **LangGraph** for tool-use workflows. - Providers: adapters for OpenAI/Anthropic/Google/Cohere/Mistral + **vLLM/Ollama** for local. - Storage: **Postgres + pgvector**, **MinIO/S3**, **Redis** for queues. - Frontend: **Next.js + Tailwind**; component library; real-time via **WebSockets**. - Observability: **OpenTelemetry** traces; run viewer.

9) Key Algorithms & Flows

9.1 Intent → TaskSpec (Normalization)

- Use a small intent parser (either rules + LLM or a fine-tuned classifier) to map free-text to:
 - `goal`, `context`, `inputs`, `constraints`, `style`, `tone`, `audience`, `format`, `acceptance_criteria`, `safety`.
- Provide smart defaults; ask *only necessary* clarifying questions.

9.2 Prompt Compilation

- Maintain prompt templates per task family with slots bound to TaskSpec.
- Support few-shot exemplars and tool calling.
- Add a **Prompt DSL** for reusable building blocks (role → steps → guardrails → tools).

9.3 Model Routing (Rule-of-Thumb + Bandits)

- Start with a rule tree (task family → candidate models) filtered by privacy/cost/latency.
- Rank candidates using **offline scores** (benchmarks per task) + **online telemetry** (win-rate, cost, latency).
- Explore/exploit via **UCB/Thompson Sampling** within user-defined budgets.

9.4 Result Verification & Self-Repair

- Build a structured rubric from acceptance criteria.
- Grade with a different model (judge) + deterministic checks (schema, coverage, references).
- If not passing, auto-generate a **repair prompt** with pinpointed gaps, then re-ask once.

10) Canonical TaskSpec (JSON Schema)

```
{
  "task_id": "uuid",
  "family": "write|code|analyze|plan|translate|summarize|rag|other",
  "goal": "string",
  "context": "string",
  "inputs": [{ "type": "text|file|url", "value": "..."}],
  "constraints": { "length": "short|medium|long", "style": ["formal",
"casual"], "tone": ["neutral", "persuasive"], "language": "en-IN",
"cost_cap_usd": 0.50, "latency_target_s": 10 },
  "audience": "string",
  "format": "markdown|html|json|code|ppt|...",
  "acceptance_criteria": ["bullet 1", "bullet 2"],
  "privacy": { "allow_provider_logging": false, "local_only": false },
  "user_prefs": { "explain_why": true, "show_alternatives": 2 }
}
```

11) API Sketch

POST /compile → TaskSpec → prompt bundle (system/instructions/user + tools)

POST /route → TaskSpec (+ prompt bundle) → {model, rationale, est_cost, est_latency}

POST /execute → {model, prompt_bundle} → streamed output + token/cost/latency metrics

POST /verify → {TaskSpec, output} → {grade, issues[], repair_prompt?}

GET /runs/:id → metadata, prompts, outputs, diffs, costs

12) UX & Screens

1. **Home / Task Wizard:** Plain-language box + toggles for cost/latency/quality; minimal questions.
 2. **Prompt Builder (Advanced):** See compiled prompt, tweak constraints, add examples.
 3. **Model Selector:** Shows recommended model + 2 alternatives with cost/latency/quality bars; manual override.
 4. **Playground:** Live run with streaming; inspector panel (tokens, latency, cost, router rationale).
 5. **Verifier Panel:** Rubric, pass/fail checks, “Repair & Re-ask” button.
 6. **Learn Center:** Micro-lessons (2–3 min), capability sheets per task family, interactive quizzes.
 7. **History & Templates:** Save/clone TaskSpecs; browse successful recipes by use-case.
-

13) Learning Layer (Curriculum)

- **Level 1:** Basics—clarifying goals, constraints, acceptance criteria.
 - **Level 2:** Task families—how prompts differ for write/code/analyze/RAG.
 - **Level 3:** Model trade-offs—cost vs quality vs latency; privacy considerations; when to go local.
 - **Level 4:** Debugging—self-repair techniques; hallucination mitigation for RAG.
 - **Artifacts:**
 - “Why this worked” explainer beside every successful run.
 - “Try this next time” tips when a run fails or needs repair.
-

14) Data, Privacy, and Safety

- **Data Minimization:** Only store what’s needed for reproducibility; opt-out of retention.
 - **PII Controls:** Detect and redact PII before provider calls if configured.
 - **Provider Policies:** Respect per-provider data usage flags; local-only mode available.
 - **Safety Filters:** Input/output moderation; restricted topics; rate limits and abuse detection.
-

15) Evaluation Plan (for A+ Grade)

1. **Gold Task Set:** 50–100 diverse, rubric-scored tasks across families.
2. **Baselines:** (a) User’s raw prompt to a single model, (b) Our compiler + router.

3. **Metrics:** Task pass-rate, edit distance to acceptance criteria, time-to-good, cost per solved task.
 4. **Human Study:** 20 participants, pairwise A/B preference (blind) for output quality & clarity.
 5. **Ablations:** Remove compiler, remove router, remove verifier—measure drops to show value.
 6. **Reporting:** Confusion matrices by family; telemetry plots (cost vs. quality); qualitative error taxonomy.
-

16) Implementation Roadmap

Week 1: Intent wizard + TaskSpec schema + basic templates (write/code/analyze).

Week 2: Router v0 (rules + user constraints) + 3 provider adapters + UI prototype.

Week 3: Verifier v0 (rubric + judge) + Repair loop + History/versioning.

Week 4: Learn Layer v0 (inline explainers) + Evaluation harness + MVP demo.

V1 (Weeks 5–8): Bandit router, RAG toolkit, team spaces, lessons catalog, analytics.

17) Risks & Mitigations

- **Shifting model landscape:** Design provider-agnostic adapters and data-driven routing.
 - **Evaluation bias (LLM-as-judge):** Use multiple judges + human audits.
 - **Hallucinations in RAG:** Cite sources; retrieval filters; verifier fact-checks.
 - **Cost spikes:** Budget caps, early stopping, draft-refine pattern.
 - **Privacy concerns:** Local-only mode; clear data controls; no training on user data by default.
-

18) Differentiators (Moat)

- **Intent→DSL compiler** producing reproducible TaskSpecs.
 - **Transparent router** with rationale & metrics (not a black box).
 - **Inline pedagogy** turning each run into a learning moment.
 - **Verifier + Repair loop** that nudges outputs to acceptance criteria.
-

19) Sample Artifacts

19.1 Example TaskSpec

```
{
  "family": "write",
  "goal": "Draft a 600-word blog on sustainable concrete for civil
contractors in the US",
  "context": "Audience: project managers; voice: practical and non-salesy;
include 3 actionable tips",
  "constraints": { "length": "medium", "language": "en-US", "cost_cap_usd":
```

```
0.15, "latency_target_s": 8 },
  "acceptance_criteria": ["3 actionable tips", "focus on US codes/standards
at a high level", "tone: practical, non-salesy"],
  "format": "markdown",
  "user_prefs": { "explain_why": true }
}
```

19.2 Router Rationale (UI snippet)

- *Chosen model*: X (reason: strong long-form coherence; meets cost cap; P95 latency 6.2s last 24h)
- *Alternatives*: Y (faster, slightly lower quality), Z (higher quality, 2× cost)

19.3 Verifier Rubric

- Tip count = 3 → pass
- Mentions US standards at high level → pass
- Tone matches “practical, non-salesy” (judge score $\geq 4/5$) → pass

20) Business & Sustainability (Optional for MP)

- **Freemium**: Limited runs/month; Pro for power features (team spaces, analytics).
- **Enterprise**: SSO, audit logs, private model hosting, on-prem connectors.
- **Education**: Learn Center with certificates; campus rollouts.

21) Demo Plan (What to show in Viva)

1. Start with a messy request, e.g., “Make a project kickoff plan for a 6-month AI app.”
2. Show TaskSpec auto-filled with clarifying defaults.
3. Show recommended model vs alternatives and router rationale.
4. Run, then open Verifier panel → pass/fail and repair suggestion.
5. Open Learn card: “Why this prompt worked.”
6. Save as template; re-use on a new but related task.

22) Conclusion

This Human↔AI Translator converts ambiguity into structured, verifiable instructions and routes work to the best-fit model—while teaching users along the way. The design is provider-agnostic, data-driven, and built for transparency. With solid evaluation and a clean demo, this project is well-positioned for an A+.